Guía de Ejercicios N°7

EIE 434 – Programación 2 | Escuela de Ingeniería Eléctrica

Unidad: 1 **Semestre:** 1er Semestre 2026 **Docente:** Guillermo Cid Ampuero

Ayudante: Amaro Díaz

1. Objetivos

Esta guía tiene como objetivo aplicar y consolidar los conceptos fundamentales de la **Programación Orientada a Objetos (POO)** en Python. Mediante la resolución de problemas contextualizados (instrumentación, sistemas multimedia y gestión universitaria), los estudiantes diseñarán jerarquías de clases, implementarán herencia y utilizarán adecuadamente atributos y métodos (de instancia, de clase y estáticos).

2. Ejercicios

Ejercicio 1 – Proyecto Integrador: Sistema de Robótica Autónoma

Usted ha sido contratado para desarrollar el software central de una nueva flota de robots industriales. Para evitar que el código se vuelva inmanejable en el futuro, deberá construir el sistema aplicando los 4 pilares de la Programación Orientada a Objetos paso a paso.

Fase 1: El Molde Base (Clases y Objetos)

a) Defina una clase `RobotMovil` que contenga un atributo de clase llamado `estado_red = 'Desconectado'`. Implemente un constructor que inicialice los atributos de instancia `nombre` y `tipo_sensor`.

b) Implemente un método `conectar_red(self)` que cambie el `estado_red` de la clase a 'Conectado' e imprima un mensaje. Agregue también un método `reportar_estado(self)` que imprima los datos del robot.

c) Pruebe su código instanciando un robot llamado 'Alpha' con sensor 'LIDAR'. Conéctelo a la red y pida su reporte.

Fase 2: Especialización (Herencia)

d) El sistema requiere robots específicos. Cree una clase heredera llamada `RobotLimpieza`. Su constructor debe utilizar la función `super()` para heredar el nombre y sensor de la clase padre.

e) Añada a `RobotLimpieza` un atributo propio llamado `capacidad_carga` inicializado en 0, y un método `recolectar_basura(self, cantidad)` que sume la cantidad a la carga actual.

Fase 3: Seguridad del Sistema (Encapsulamiento)

f) Se detectó que un usuario podría ingresar valores negativos de basura. Modifique `RobotLimpieza` para que la carga sea privada (`__capacidad_carga`).

g) Cree los métodos `get_carga(self)` y `set_carga(self, nueva_carga)`. El setter debe bloquear cualquier intento de asignar una carga menor a 0 o mayor a 100.

h) Actualice su método `recolectar_basura` para que pase obligatoriamente por el getter y el setter. Intente recolectar 150 unidades para comprobar que la seguridad funciona.

Fase 4: Arquitectura Escalable (Abstracción y Polimorfismo)

i) Para que el sistema no colapse a futuro, convierta `RobotMovil` en una clase abstracta (importando `ABC` y `abstractmethod`).

j) Agregue a la clase padre un método abstracto vacío llamado `procesar_entorno(self)`. Asegúrese de que su `RobotLimpieza` cumpla el contrato implementando este método (ej. imprimiendo 'Aspirando suelo').

k) Cree una nueva clase `RobotVigilancia` que herede de `RobotMovil`, con un atributo `camara_activa = True` y que también cumpla el contrato implementando su propia versión de `procesar_entorno(self)`.

l) **Prueba Final (Polimorfismo):** Cree una lista llamada `flota` que contenga un robot de limpieza y uno de vigilancia. Recorra la lista con un `for` y ejecute `procesar_entorno()` para cada uno.

Nota: Solución de la Clase: Repositorio del Curso